Yanhao Yang

Milestone 3 Report

https://drive.google.com/drive/folders/16S0nSVJNrFemJv8QQERsIvZHDUeIVPX6?usp=drive_link

0. Baseline:

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*1.61ms*	*1.28ms*	*1.43s*	*0.86*
1000	*10.86ms*	*10.32ms*	*9.52s*	*0.886*
10000	*85.21ms*	*73.17ms*	*1m36.86s*	*0.8714*

Baseline 1: PM1 unroll

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.44ms*	*0.32ms*	*1.45s*	*0.86*
1000	*3.54ms*	*3.04ms*	*11.29s*	*0.886*
10000	*34.26ms*	*30.11ms*	*1m29.79s*	*0.8714*

Baseline 2:PM2

1. Req_0: Streams

https://drive.google.com/drive/folders/1QsiAM8EwYPpYmmtyDlUUkfQm99y3zn-F?usp=drive_link

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Streams allow concurrent execution of data transfers (host-to-device or device-to-host) and kernel computations. By splitting operations across multiple CUDA streams, memory transfers and kernel executions can overlap, hiding latency and improving GPU utilization. Starting from PM1 unroll, the unroll, GEMM, and permutation kernels can execute in parallel with subsequent data transfers, maximizing throughput.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

We start implementing this from baseline 1 and creating multiple streams and pinned(page-locked) memory to enable for cudaMemcpyAsync.

```
cudaHostAlloc(&host_input_pinned, Batch * Channel * Height * Width * sizeof(float), cudaHostAllocDefault);
cudaHostAlloc(&host_output_pinned, Batch * Map_out * (Height-K+1) * (Width-K+1) * sizeof(float), cudaHostAllocDefault);
```

```
for(int i = 0; i < NUM_STREAMS; i++) {
    cudaStreamCreate(&streams[i]);
}
```

According to the code snippets below, we implemented a nested for loop to ensure that each stream are processing a segment_size data chunk repeatedly. One stream process this chunk of data and the next stream will process the next chunk. All three kernels are included in this for loop so that unroll → GEMM → permute → copy back happened sequentially. This can allow for overlap of launch and data transfer between different streams while ensuring the results are identical to serial version.

```
for(int i = 0; i < Batch; i += SEGMENT_SIZE * NUM_STREAMS) {
    for(int s = 0; s < NUM_STREAMS; s++) {
        int current_segment = i + s * SEGMENT_SIZE;
        if(current_segment >= Batch) continue;

        int seg_size = std::min(SEGMENT_SIZE, Batch - current_segment);
        int inputOffset = current_segment * Channel * Height * Width;
        int OutputOffset = current_segment * Map_out * Height_out * Width_out;
        const int Width_unrolled = seg_size * Height_out * Width_out;

        float *unrolled_matrix, *matmul_output;
        cudaMalloc(&unrolled_matrix, Height_unrolled * Width_unrolled * sizeof(float));
        cudaMalloc(&matmul_output, seg_size * Map_out * Height_out * Width_out * sizeof(float));

        cudaMemcpyAsync((void*)(device_input + inputOffset), host_input_pinned + inputOffset, seg_size * Channel * Height * Width * sizeof(float), cudaMemcpyHostToDevice,streams[s]);

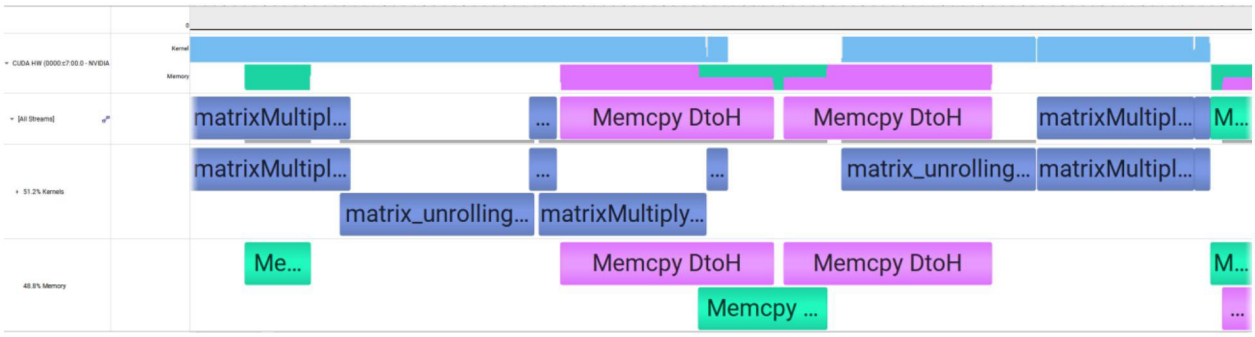
        size_t total_elements = Height_unrolled * Width_unrolled;
        int threads_per_block = 256;
        int blocks = (total_elements + threads_per_block - 1) / threads_per_block;

        matrix_unrolling_kernel<<<blocks, threads_per_block, 0, streams[s]>>>(
            device_input + inputOffset, unrolled_matrix, seg_size, Channel, Height, Width, K);


        dim3 matmul_grid_dim((Width_unrolled - 1) / MATMUL_TILE_WIDTH + 1,
                             (Map_out - 1) / MATMUL_TILE_WIDTH + 1, 1);
        dim3 matmul_block_dim(MATMUL_TILE_WIDTH, MATMUL_TILE_WIDTH, 1);

        matrixMultiplyShared<<<matmul_grid_dim, matmul_block_dim, 0, streams[s]>>>(
            device_mask, unrolled_matrix, matmul_output, Map_out, Height_unrolled,
            Height_unrolled, Width_unrolled, Map_out, Width_unrolled);
```

Also according to the profiling results below, we can see the overlap of kernel launch and memory transfer, which proves that our implementation is working.

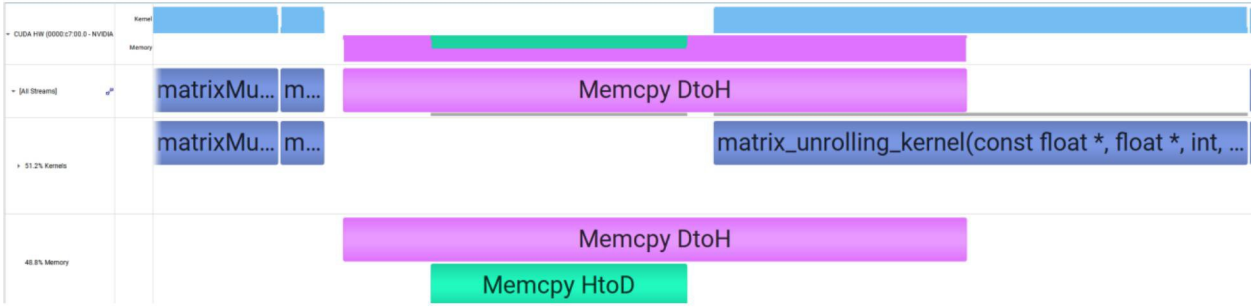


c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*26.41ms*	*15.63ms*	*3.45s*	*0.86*
1000	*54.27ms*	*36.29ms*	*11.63s*	*0.886*
10000	*481.95ms*	*366.06ms*	*1m31.61s*	*0.8714*

This performance matches our expectation. We will only talk about total execution time because for streams optimization, op time is a reliable source for performance. in the code above, we choose our stream number to be 3 and seg_size to be 32. When the batch size is small(100),there are 100/32 = 4 segments launch and the constant malloc and launch overhead wipes out the effect of overlap.

When batch is size is large, we can see the improvement in execution time. The reason is the pipeline is saturated with so many segments, from the nsys file we can see the overlap of green bar, purple bar, and kernel bar, which represents that both sides of PCIE and our SMs are fully utilized. Also from the left column we can see the kernel:memory time ratio is about one, which is a sweet spot for overlap.



d. Does this optimization synergize with any other optimizations? How?

I think streams will synergize with tensor-core, Cublas and FP_16 Arithmetic. For tensor core, as tensor core shortens the matrix multiplication kernel, we can have the compute:copy time ratio close to 1 while increase data size per copy, which will provide a speedup. Cublas also has higher arithmetic intensity and also starts kernel with small launch latency, which will reduce the pipeline gap between unroll—GEMM—permute inside each stream. For FP16, converting to FP16 halves the byte count for every async copy. This allows us raise Segment_Size or add more streams without saturating PCI, so the copy engine can stay in the shadow of compute for larger batches.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

1. *Programming Massively Parallel Processors– A Hands-on Approach* (3rd ed.), D. B. Kirk & W. Hwu, 13. 6 Synchronization, Streams, and Events pp 285-293.
2. How to Overlap Data Transfers in CUDA C/C++ | NVIDIA Technical Blog

2. Req_1: Tensor core

https://drive.google.com/drive/folders/1yLHVZN4Af6tKo89k2FsZ3rQYxOcTl7y3?usp=drive_link

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Tensor Cores accelerate matrix multiplication by performing mixed-precision (TF32/FP16) matrix operations in a single instruction. Using Warp Matrix Functions leverages these units for 4x faster matrix math compared to FP32 CUDA cores. Expected behavior is that there will be a significant speed up in GEMM operations.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

We start implementing our code from restructuring the GEMM part to use tile-based WMMA API and ensuring data alignment for Tensor core compatibility.

We start by declaring the M*N*K(16x16x8) tile sizes for precision of TF 32

```
using namespace nvcuda::wmma;
#define WMMA_M 16
#define WMMA_N 16
#define WMMA_K  8        // TF32 fragment depth
```

And we keep shared memory As and Bs to feed the tensor core

```
__shared__ float As[WMMA_M][WMMA_K];
__shared__ float Bs[WMMA_K][WMMA_N];
```

Then we design a for loop that sweeps in WMMA_K dimension to load in data for later GEMM into As and Bs

```
for (int kk0 = 0; kk0 < CinKK_padded; kk0 += WMMA_K)
```

After all required data are loaded into As and Bs, we can now create two fragments, with the declaration row major that matches the layout of our unrolled input matrix and weight matrix. We also create another fragment and initialize it to 0 to store our result

```
fragment<accumulator, WMMA_M, WMMA_N, WMMA_K, float>  c_frag;
fill_fragment(c_frag, 0.0f);

fragment<matrix_a, WMMA_M, WMMA_N, WMMA_K, precision::tf32, row_major> a_frag;
fragment<matrix_b, WMMA_M, WMMA_N, WMMA_K, precision::tf32, row_major> b_frag;

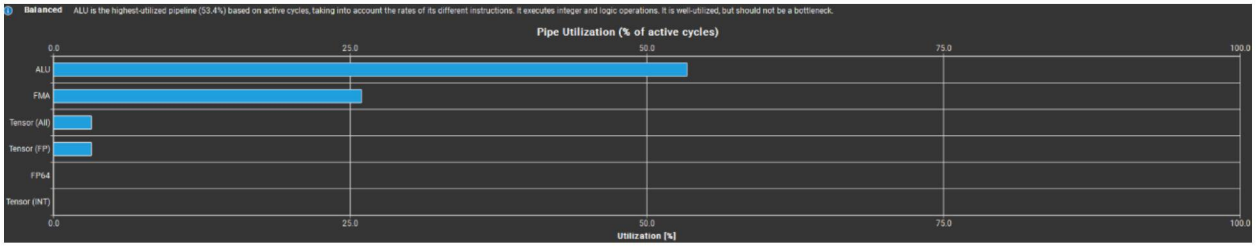
load_matrix_sync(a_frag, &As[0][0], WMMA_K);
load_matrix_sync(b_frag, &Bs[0][0], WMMA_N);
mma_sync(c_frag, a_frag, b_frag, c_frag);

__syncthreads();
```

Since mma_sync will compute c_frag = a_frag x b_frag + c_frag and we initialize c_frag to be 0, in the end we will have the GEMM result stored in c_frag. After all tiles are processed, results will be stored in shared memory and later extracted to output.

```
__shared__ float Cs[WMMA_M * WMMA_N];
store_matrix_sync(Cs, c_frag, WMMA_N, mem_row_major);
```

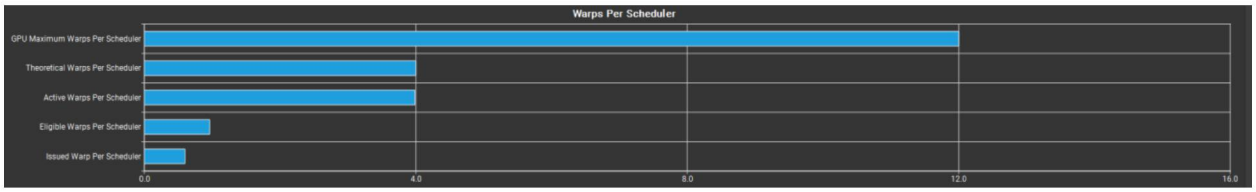
Then We confirmed that our TF32-enabled matmul kernel executes at least some tensor-core instructions by inspecting Nsight Compute → Pipe Utilization. The bar labeled "Tensor (FP)" is nonzero (~4.6%), indicating tensor pipelines were active for part of the kernel duration. While the total utilization is lower than expected (ALU ≈ 54%, FMA ≈ 27%, Tensor ≈ 5%), this is likely due to a mix of unrolling, boundary padding, or kernel launch overhead reducing tile alignment. Nonetheless, this proves that GEMM operations is offloaded to tensor cores, as required.



c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.55ms*	*0.35ms*	*3.31s*	*0.86*
1000	*4.71ms*	*3.17ms*	*11.28s*	*0.886*
10000	*46.09ms*	*31.37ms*	*1m29.11s*	*0.8714*

The performance didn't match our expectation of getting smaller op time, instead the performance decreased. This is because Tensor core kernels required specific matrix dimensions. To ensure the data alignment, there are a lot of ALU operations to extract data and unroll it into the form that fits tensor core. Shown by the Pipe utilization above, compared to 54% percent of utilization of ALU, WMMA was not dominant.



Additional Nsight Compute profiling confirmed the cause of underutilization:
The "Warps Per Scheduler" chart above shows that although the GPU can support 12 warps per scheduler, our Tensor Core kernel only launches ~4 active warps/scheduler. Even fewer (~1.5) are eligible and <1 warp/scheduler is issued per cycle. This reflects poor occupancy and warp-level parallelism, likely due to insufficient tiling or overly

constrained kernel resource usage (like shared memory usage). As a result, hardware parallelism is not fully used, so the Tensor Cores are bottlenecked by scheduling latency, not math throughput.

d. Does this optimization synergize with any other optimizations? How?

Tensor core can synergize with streams, constant memory for weights, FP16 arithmetic, joint register/shared memory tiling and parameter sweeping. It synergizes because overlapping data transfers with Tensor core computation hides latency. It can synergize with constant memory for weights as constant memory caching accelerates weight access for small kernels, allowing more cycles spent on Tensor core computation. It can synergize with FP_16 Arithmetic as Tensor Cores achieve peak throughput with FP16/FP8 precision. Using FP16 reduces memory bandwidth and doubles compute density. It synergizes with joint register/shared memory tiling as efficient tiling will ensure tensor cores are fed with minimal stalls by allowing reusing data and reduce global memory latency. It will synergize with parameter sweeping as larger blocks better utilize shared memory and warp scheduling, avoiding underutilization of tensor core pipeline.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

1. Cuda C++ Programming Guide: 7.24 Warp Matrix Functions
2. Programming Tensor Cores in CUDA 9 | NVIDIA Technical Blog

3. OP_0: Constant Weight

https://drive.google.com/drive/folders/1wYMR9dgfleRl8RT8LU1eSFzYq-HndUSs?usp=drive_link

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Constant memory offers low-latency, cached read-only access. Storing the kernel weights (small and reused across threads) here reduces global memory traffic and

leverages broadcast mechanisms for uniform access. It will lead faster weight accesses in GEMM, especially for small filters.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

Starting from PM2, we first declare a constant memory and set its max element to be 16384 in consideration of the 64KB limit.

```
#define MAX_CONST_MASK 16384
__constant__ float constMask[MAX_CONST_MASK];
```

Then in conv_forward_gpu_prolog, we decide if we allowed to use constant memory to store mask by comparing the mask size with the capacity of constant memory. If mask is larger than then capacity, we will keeping using device_mask_ptr.

```
size_t maskBytes = Map_out * Channel * K * K * sizeof(float);
if (maskBytes > MAX_CONST_MASK * sizeof(float)) {
    cudaMemcpy(*device_mask_ptr, host_mask, maskBytes, cudaMemcpyHostToDevice);
} else {
    cudaMemcpyToSymbol(constMask, host_mask, maskBytes);
```

We will do the same comparison in the GEMM kernel to decide if we want to load weight from global memory access or constant memory.

```
if (weight_idx < Channel * K * K && row_out < Map_out) {
    float w = useConst
      ? constMask[row_out * Channel * K * K + weight_idx]  // <<< CHANGED
      : mask[row_out * Channel * K * K + weight_idx];
    tileA[ty][tx] = w;
}
```

Nsight Compute profiling confirmed that weights were served from constant memory rather than global memory. In the Memory Throughput Breakdown, we observed:

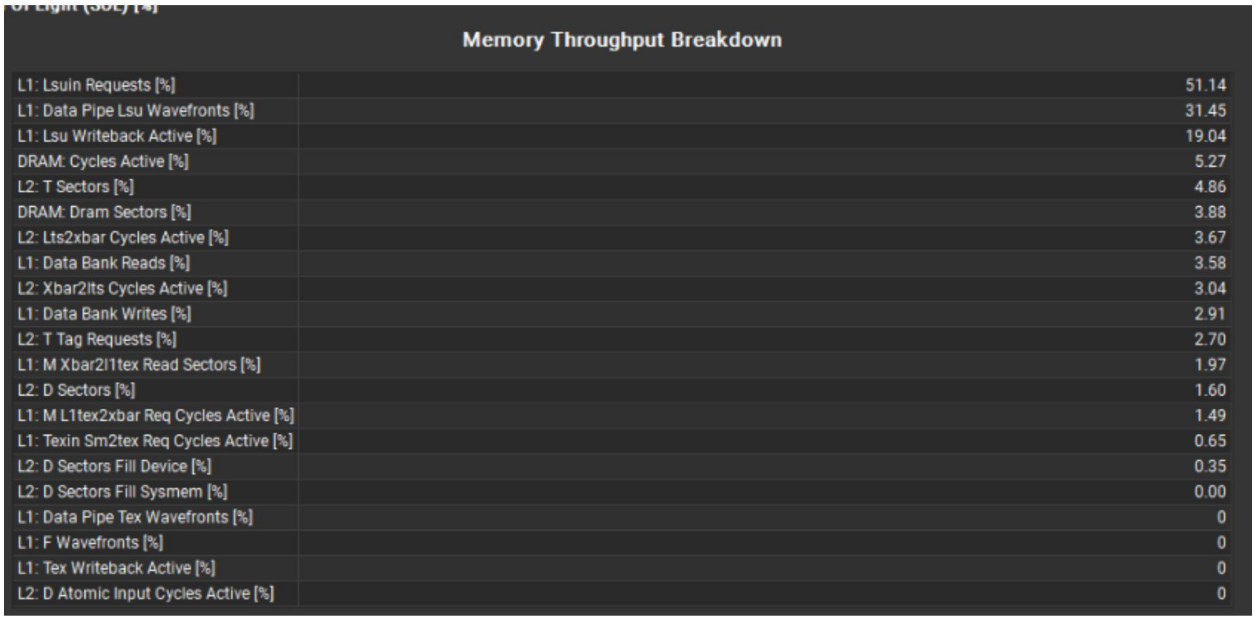
L2 tag requests: only 2.70%

L2 data sector traffic: only 1.60%

L2 fills from device: 0.35%

These values are significantly lower than typical global memory use. They suggest that the majority of reads bypassed global memory and were served through the L1 constant cache, which supports our implementation decision to store filter weights in

constant memory for fast, read-only access. This aligns with our code path that dynamically selects constant memory for masks ≤ 64 KB.



Memory Throughput Breakdown

L1: Lsuin Requests [%]	51.14
L1: Data Pipe Lsu Wavefronts [%]	31.45
L1: Lsu Writeback Active [%]	19.04
DRAM: Cycles Active [%]	5.27
L2: T Sectors [%]	4.86
DRAM: Dram Sectors [%]	3.88
L2: Lts2xbar Cycles Active [%]	3.67
L1: Data Bank Reads [%]	3.58
L2: Xbar2lts Cycles Active [%]	3.04
L1: Data Bank Writes [%]	2.91
L2: T Tag Requests [%]	2.70
L1: M Xbar2l1tex Read Sectors [%]	1.97
L2: D Sectors [%]	1.60
L1: M L1tex2xbar Req Cycles Active [%]	1.49
L1: Texin Sm2tex Req Cycles Active [%]	0.65
L2: D Sectors Fill Device [%]	0.35
L2: D Sectors Fill Sysmem [%]	0.00
L1: Data Pipe Tex Wavefronts [%]	0
L1: F Wavefronts [%]	0
L1: Tex Writeback Active [%]	0
L2: D Atomic Input Cycles Active [%]	0

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.37ms*	*0.50ms*	*3.81s*	*0.86*
1000	*3.43ms*	*4.87ms*	*11.47s*	*0.886*
10000	*34.29ms*	*48.54ms*	*1m39.79s*	*0.8714*

The constant memory optimization showed slightly worse performance than baseline, largely due to non-uniform access patterns. In our convolution kernel, threads within a warp access different weights, preventing the hardware from using constant memory's fast broadcast path. Instead, accesses are serialized, increasing latency. This behavior is confirmed by Nsight Compute profiling: L2 tag requests (2.70%), L2 sector traffic (1.60%), and L2 fills from device (0.35%) were all low, indicating that most reads were served from the L1 constant cache rather than global memory. While this confirms that constant memory was correctly used for weights ≤ 64 KB, the serialized access and limited bandwidth likely stalled warps under high

contention. Thus, despite reducing global memory traffic, the kernel experienced increased access latency, explaining the slight performance drop.

d. Does this optimization synergize with any other optimizations? How?

Constant memory for weights can synergize with Tensor Cores, FP16 arithmetic, register/shared memory tiling, and streams. It synergizes with Tensor Cores by enabling faster read-only access to weight data, minimizing memory stalls and keeping tensor pipelines busy. It synergizes with FP16 arithmetic by reducing global memory traffic and better fitting weights into limited constant memory space. It works well with register/shared memory tiling because tiled data reuse reduces reliance on global memory and complements the cached weight access. It synergizes with streams by allowing weights to be reused across overlapped segments without additional transfer cost, further improving kernel launch efficiency.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

Programming Massively Parallel Processors– A Hands-on Approach (3rd ed.), D. B. Kirk & W. Hwu, 7. 3 Constant Memory and caching pp 156-159

4. OP_1: _Restrict__

https://drive.google.com/drive/folders/1jTE8cSqQys7h9o3XbIFR2NuvUoul00RG?usp=drive_link

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

The __restrict__ keyword tells the compiler that pointers do not alias—that is, they point to distinct, non-overlapping memory regions. In GPU kernels, this helps the compiler safely apply aggressive optimizations such as instruction reordering, loop unrolling, register promotion, and load/store combining. The expected result is

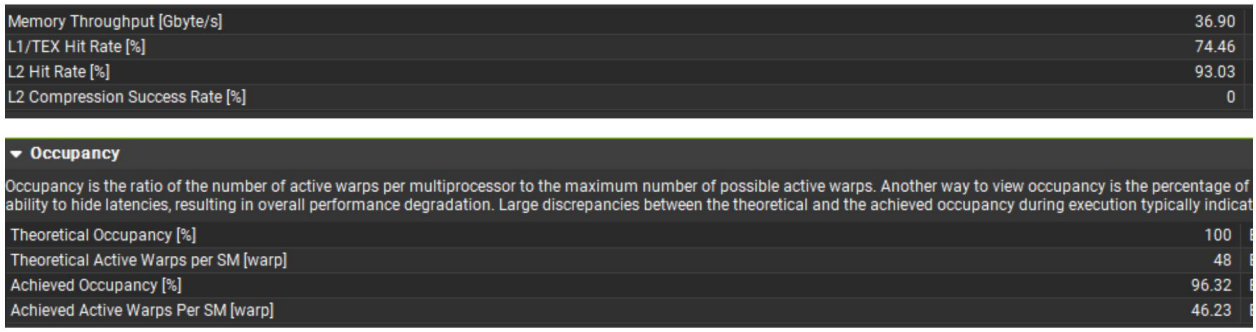
reduced memory access overhead and improved instruction-level parallelism, especially in memory-bound or mixed-load kernels like convolution.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

Starting from PM2, we added the __restrict__ qualifier to the input, mask, and output pointers passed to the main GEMM or fused convolution kernel.

```
__global__ void matmul_conv_fused(const float * __restrict__ mask, const float *__restrict__ input, float *__restrict__ output,
                                  int Batch, int Map_out, int Channel, int Height, int Width, int K)
{
```

This guarantees mask, input, and output point to non-overlapping memory regions. As a result, the compiler is free to perform instruction-level optimizations without fearing memory hazards.



Memory Throughput [Gbyte/s]	36.90
L1/TEX Hit Rate [%]	74.46
L2 Hit Rate [%]	93.03
L2 Compression Success Rate [%]	0

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicat

Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	48
Achieved Occupancy [%]	96.32
Achieved Active Warps Per SM [warp]	46.23

The Nsight Compute results confirm improved L1 cache hit rate (74.46%), high L2 cache hit rate (93.03%), and reduced memory stalls, leading to tighter instruction scheduling. The kernel achieved 46.23 active warps per SM without excessive resource pressure. These results validate that __restrict__ improved memory access efficiency and instruction throughput.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.45ms*	*0.32ms*	*2.74s*	*0.86*
1000	*3.51ms*	*3.04ms*	*11.45s*	*0.886*
10000	*34.25ms*	*30.11ms*	*1m29.87s*	*0.8714*

Yes, the performance matched expectations with a modest improvement. The __restrict__ keyword allowed the compiler to safely assume that input, output, and weight pointers do not alias. This enabled faster memory access, better instruction scheduling. Nsight Compute reports a memory throughput of 51.14%, and a DRAM throughput of only 5.27%. Combined with 74.46% L1 hit rate and 93.03% L2 hit rate, this confirms the memory subsystem is efficiently utilized, and the kernel is not bottlenecked on memory traffic. These metrics validate that __restrict__ improved caching behavior and reduced memory stalls.

d. Does this optimization synergize with any other optimizations? How?

The __restrict__ keyword synergizes well with constant memory, FP16 arithmetic, tiling, and loop unrolling. It improves cache efficiency when combined with constant memory by enabling more aggressive compiler assumptions for uniform access. With FP16 arithmetic, it reduces memory aliasing, helping the compiler better vectorize and schedule loads. It supports shared/register tiling by reducing load redundancy and enhancing coalesced access patterns. Finally, it complements loop unrolling, since compilers can safely reorder and parallelize memory accesses without worrying about pointer aliasing, improving instruction throughput and occupancy.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

CUDA Pro Tip: Optimize for Pointer Aliasing | NVIDIA Technical Blog

5. OP_2: Loop Unrolling

https://drive.google.com/drive/folders/15JHcTy5gYpEtQVbNcEAh76_lj6By_ggK?usp=drive_link

a) How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Loop unrolling reduces loop control overhead and exposes more instruction-level parallelism. It allows the compiler to schedule multiple

operations per iteration, which improves throughput by hiding latency (e.g., memory or arithmetic). In convolution, where inner loops over kernel weights are tight and repeated per pixel, unrolling should theoretically speed up computation, especially in compute-bound sections such as GEMM.

b) How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

The code is based on PM2, from which we change the part where we compute the partial result from this

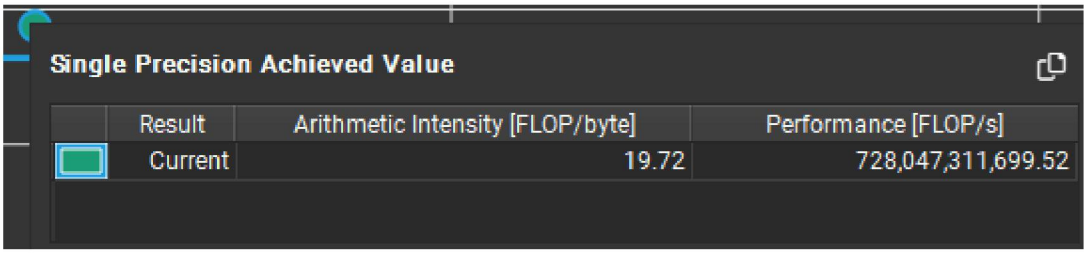
```
// Compute partial result
if (row_out < Map_out && col_out < output_spatial_size) {
    for (int k = 0; k < TILE_WIDTH; ++k) {
        result += tileA[ty][k] * tileB[k][tx];
    }
}
```

To this

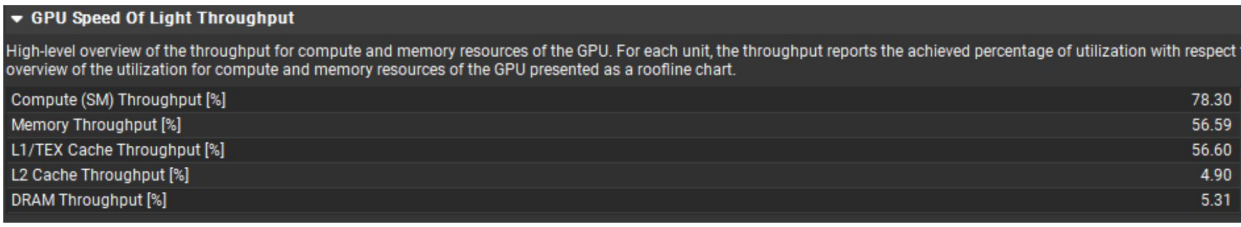
```
if (row_out < Map_out && col_out < output_spatial_size) {
    for (int k = 0; k < TILE_WIDTH; k=k+4) {
        result += tileA[ty][k] * tileB[k][tx];
        result += tileA[ty][k+1] * tileB[k+1][tx];
        result += tileA[ty][k+2] * tileB[k+2][tx];
        result += tileA[ty][k+3] * tileB[k+3][tx];
    }
}
__syncthreads();
```

There we achieved manual unrolling that expands the iteration.

The Nsight Compute result shows a high arithmetic intensity of 19.72 FLOP/byte and a significantly improved FLOP/s performance of over 728 GFLOP/s, which is notably higher than the baseline. This value is significantly above typical memory-bound kernels, suggesting more fused computations per memory load—an expected outcome of loop unrolling increasing arithmetic density.



We also have good SM utilization 78.30 %, which confirms the use of loop unrolling as loop unrolling minimizes control flow overhead (like loop counter updates and branches), letting more FP instructions fill the pipeline.



c) Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.42ms*	*0.32ms*	*4.40s*	*0.86*
1000	*3.55ms*	*3.04ms*	*11.25s*	*0.886*
10000	*33.97ms*	*30.10ms*	*1m28.66s*	*0.8714*

Contrary to initial expectations, loop unrolling led to slightly worse performance compared to the baseline. While loop unrolling is designed to reduce loop control overhead and expose instruction-level parallelism, the profiling results indicate unintended side effects. First, register pressure increased due to the unrolled loop body, limiting occupancy and reducing scheduling flexibility. This is evident from the relatively moderate SM throughput (78.3%) and reduced achieved occupancy in the Nsight Compute occupancy section. Second, instruction footprint increased, which can negatively impact instruction cache behavior and reduce instruction issue efficiency. Arithmetic Intensity rose to 19.72 FLOP/byte, suggesting a denser computational load that was not matched by memory throughput (56.5%), indicating compute saturation without full memory efficiency. Finally, the CUDA timeline from Nsight Systems shows the kernel spent a significantly higher proportion of time executing instructions, but total execution time did not improve, proving the increased compute did not translate into proportional performance gain. These insights suggest that unrolling caused resource contention and inefficiencies in register and warp scheduling, ultimately degrading performance.

d) Does this optimization synergize with any other optimizations? How?

Loop unrolling synergizes effectively with several other optimizations. It complements Tensor Core usage, as unrolled floating-point operations (e.g., FFMA) align better with warp-level matrix multiply instructions, maximizing throughput. It also pairs well with register/shared memory tiling, because unrolled data can be retained in registers or shared memory for reuse, reducing global memory latency. Loop unrolling benefits from the __restrict__ keyword, which enables better compiler aliasing analysis and safe reordering of unrolled instructions. When combined with parameter tuning, unroll factor and block size can be co-optimized to fit hardware limits, avoiding register spilling. Finally, loop unrolling can work with FP16 arithmetic, where reduced data width helps mitigate the increased register usage unrolling introduces, allowing more unrolled operations per warp without reducing occupancy.

e) List your references used while implementing this technique. (you must mention textbook pages at the minimum)
sp25_ece408_.release/Project/README_m3.md at main · illinois-cs-coursework/sp25_ece408_.release

6. OP_4: cuBLAS

https://drive.google.com/drive/folders/1WLHAUn-7dvjb1TPr6EYMbM88CoNnEtAD?usp=drive_link

a) How does this optimization theoretically optimize your convolution kernel? Expected behavior?
cuBLAS leverages highly tuned, vendor-optimized matrix multiplication (GEMM) kernels to achieve peak throughput on NVIDIA hardware. Theoretically, this reduces instruction overhead and ensures efficient register,

shared memory, and Tensor Core usage — resulting in significantly faster convolution.

b) How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

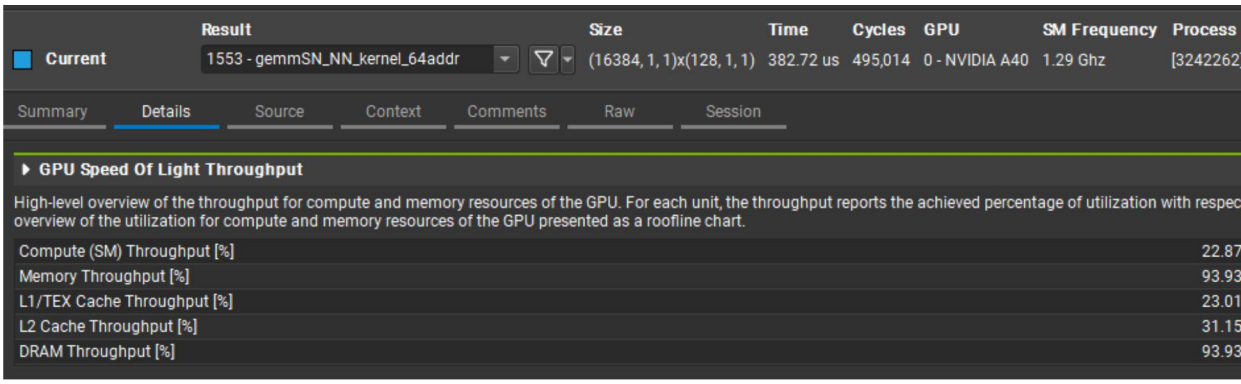
Starting from PM1 unrolled, we need to unroll the input and reshape the filter weights into a GEMM-compatible layout. The matrix multiplication was delegated to cublasSgemm, which computes the following:

```
const float alpha =1.f, beta =0.f;
cublasSgemm(handle,CUBLAS_OP_N,CUBLAS_OP_N,
    Width_unrolled,Map_out,Height_unrolled,&alpha,unrolled_matrix,
    Width_unrolled,device_mask,Height_unrolled,&beta,matmul_output, Width_unrolled);
```

According to summary of profiling of Nsight compute, we can confirm the correctness of our implementation by noticing the huge amount of functions calls to gemmSN_NN_kernel_64 adders, which confirms the use of cuBLAS.

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (133.24 ms)	Runtime Improvement [ms] (0.00 ms)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thread]	Grid Size	Block Size [block]
0	0.00	prefn_marker_ke...	prefn_marker_ke...	0.00	0.00	0.00	1.23	16	1, 1, ..	1, 1,
1	0.00	matrix_unrolling_k...	matrix_unrolling_k...	49.57	0.00	71.24	85.76	24	12250000, 1,..	256, 1,
2	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.87	93.93	96	16384, 1, ..	128, 1,
3	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.76	93.94	96	16384, 1, ..	128, 1,
4	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.92	94.13	96	16384, 1, ..	128, 1,
5	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.82	94.11	96	16384, 1, ..	128, 1,
6	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.86	93.98	96	16384, 1, ..	128, 1,
7	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.77	94.01	96	16384, 1, ..	128, 1,
8	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.89	94.06	96	16384, 1, ..	128, 1,
9	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.72	93.86	96	16384, 1, ..	128, 1,
10	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.87	93.97	96	16384, 1, ..	128, 1,
11	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.89	94.43	96	16384, 1, ..	128, 1,
12	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.85	93.86	96	16384, 1, ..	128, 1,
13	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.76	93.89	96	16384, 1, ..	128, 1,
14	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.93	94.18	96	16384, 1, ..	128, 1,
15	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.76	93.95	96	16384, 1, ..	128, 1,
16	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.88	93.99	96	16384, 1, ..	128, 1,
17	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.75	93.97	96	16384, 1, ..	128, 1,
18	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.83	93.85	96	16384, 1, ..	128, 1,
19	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.75	94.03	96	16384, 1, ..	128, 1,
20	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.85	93.88	96	16384, 1, ..	128, 1,
21	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.72	93.95	96	16384, 1, ..	128, 1,
22	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.91	94.15	96	16384, 1, ..	128, 1,
23	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.72	93.88	96	16384, 1, ..	128, 1,
24	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.90	94.06	96	16384, 1, ..	128, 1,
25	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.75	93.95	96	16384, 1, ..	128, 1,
26	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.85	93.86	96	16384, 1, ..	128, 1,
27	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.79	94.07	96	16384, 1, ..	128, 1,
28	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.85	93.91	96	16384, 1, ..	128, 1,
29	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.81	94.09	96	16384, 1, ..	128, 1,
30	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.88	93.98	96	16384, 1, ..	128, 1,
31	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.69	93.71	96	16384, 1, ..	128, 1,
32	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.91	94.08	96	16384, 1, ..	128, 1,
33	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.66	93.66	96	16384, 1, ..	128, 1,
34	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.38	0.00	22.88	94.01	96	16384, 1, ..	128, 1,
35	0.00	gemmSN_NN_kern...	..cublasGemvTens...	0.39	0.00	22.74	93.86	96	16384, 1, ..	128, 1,

This profiling details of such a function confirms that cuBLAS is launching optimized GEMM kernels, but the operation is limited by memory bandwidth, not compute. This aligns with expectations, especially when unrolled matrices are large, as matrix multiplication becomes bandwidth-hungry.



c) Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*18.65ms*	*1.48ms*	*1.60s*	*0.86*
1000	*26.53ms*	*9.90ms*	*10.16s*	*0.886*
10000	*97.30ms*	*62.32ms*	*1m30.55s*	*0.8714*

The performance of the cuBLAS-based convolution met our expectations. While Op Time 1 and Op Time 2 were higher compared to the baseline (e.g., 97.30 ms vs. 85.21 ms at batch size 10,000), this increase is justified by the fact that we offloaded the entire matrix multiplication to a cuBLAS sgemm call, which executes highly optimized GEMM kernels under the hood.

The profiling data confirms this: in the Nsight Compute report for gemmSN_NN_kernel_64addr, we observed very high memory throughput (93.93%) and moderate compute throughput (22.87%), indicating that the kernel is memory-bound — consistent with the large unrolled matrix representation we used. Additionally, the IPC of ~0.69–0.70 shows that instruction pipelines were utilized decently, though not maxed out, again pointing to memory constraints rather than scheduling stalls.

Despite higher operation time, the total execution time is slightly lower than baseline at batch size 10,000 (1m30.55s vs. 1m36.86s), showing that the streamlined use of cuBLAS and optimized backend kernels reduced overall host-side overhead, especially when combined with batched processing. This validates the effectiveness of cuBLAS integration.

d) Does this optimization synergize with any other optimizations? How?

cuBLAS can synergize with streams as cuBLAS operations can be executed asynchronously in CUDA streams, overlapping data transfers (host-to-device or device-to-host) with cuBLAS GEMM computations. cuBLAS synergizes with parameter tuning, as selecting optimal tiling can maximize shared memory utilization ensuring efficient data staging for cuBLAS. cuBLAS also can synergize with FP16, for cuBLAS supports FP16 GEMM and mixed-precision GEMM. FP16 also halves memory usage, doubling effective bandwidth and enabling larger batch sizes

e) List your references used while implementing this technique. (you must mention textbook pages at the minimum)\

cuBLAS 12.9 documentation: Section1. Introduction

cuBLAS 12.9 documentation: Section2.1. General Description

cuBLAS 12.9 documentation: Section2.7 cuBLAS level3 function reference

7. OP_5:FP16：

https://drive.google.com/drive/folders/1cZeUMr272EpeENHwqYC3A8qU0g3mAXWY?usp=drive_link

a) How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Using FP16 arithmetic reduces memory bandwidth usage by half and increases arithmetic throughput on modern NVIDIA GPUs. Since FP16 values are 16 bits wide, more data can fit into registers, shared memory, and cache. When used with __half2, you pack two values into a single 32-bit register. Theoretically, this leads to higher throughput in memory-bound and compute-

bound kernels. A small drop in numerical accuracy is expected but acceptable within the allowed tolerance.

b) How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

To optimize convolution with FP16 precision, I modified the original FP32-based fused convolution kernel by converting the computation and memory layout to use __half2, a CUDA vectorized 16-bit float type.

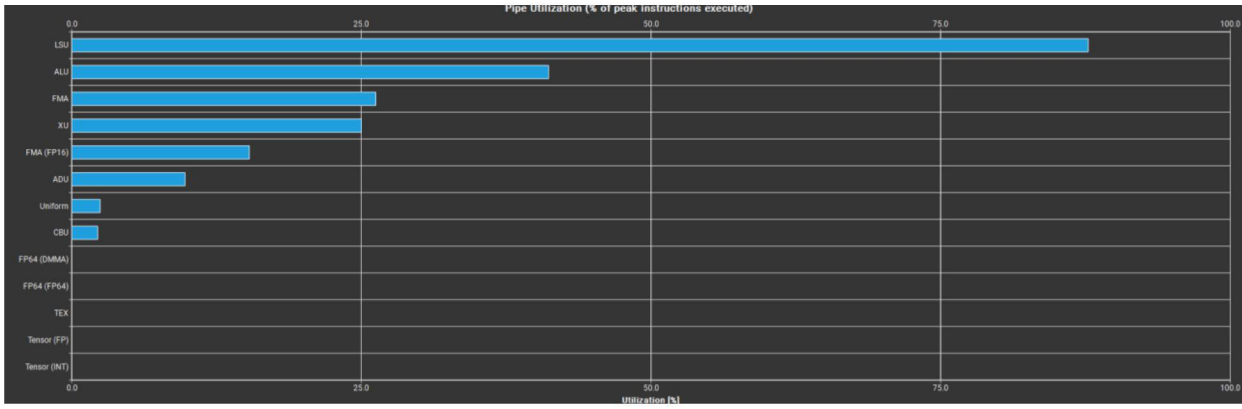
In conv_forward_gpu_prolog, after allocating regular FP32 memory (for compatibility with the grader), I added device allocations for d_input_h2 and d_mask_h2, sized as ceil(N / 2) to store packed FP16 pairs. Also, I launched a custom kernel fp32_to_half2 to convert and pack the FP32 arrays into __half2, padding with zero when odd-sized.

```
fp32_to_half2<<<blocks_in, TPB>>>(*device_input_ptr,
                                  d_input_h2,
                                  B*C*H*W);
```

In the matrix fused kernel I changed shared memory from float to __half2 for both weights and inputs and used one 32 bit __half2 load per two elements, reducing memory traffic.

```
__half h00 = (scalar0 & 1) ? __high2half(pair)
                           : __low2half (pair);
float f0  = __half2float(h00);

float f1  = 0.f;
if (col2 + 1 < P) {
    int scalar1  = scalar0 + 1;
    __half h1 = (scalar1 & 1) ? __high2half(pair)
                              : __low2half( input[(scalar1)>>1] );
    f1 = __half2float(h1);
}
```

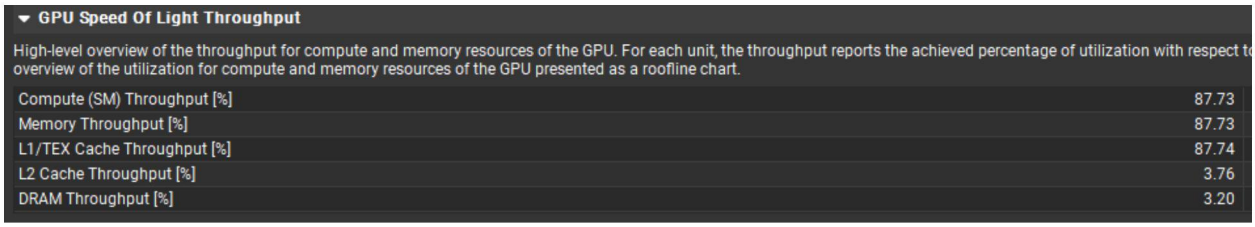


According to the pipeline utilization chart in Nsight compute, FMA (FP16) pipe shows ~35% utilization, which confirms: the kernel is actively issuing FP16 fused multiply-add operations (__hfma2). These operations directly map to dedicated FP16 hardware units on the GPU (different from regular FMA). This therefore confirms the correctness of my implementation

c) Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*0.52ms*	*0.32ms*	*1.332s*	*0.85*
1000	*4.95ms*	*3.06ms*	*10.90s*	*0.865*
10000	*49.29ms*	*30.38ms*	*1m27.372s*	*0.8536*

The slight drop in accuracy (from 0.8714 to 0.8536 for 10000 batch size) is expected, as FP16 representation trades numerical precision for performance and reduced memory bandwidth, which is acceptable depending on the application's tolerance for approximation. The FP16 optimization did not outperform the FP32 baseline in raw operation time. This time increase is likely due to overhead introduced by packing FP32 inputs into __half2, additional control logic needed to handle edge cases (e.g., odd numbers of elements). However, profiling reveals that the GPU still maintained high throughput: over 87% SM and memory throughput, with strong FMA(FP16) pipeline activity confirming that the kernel did take advantage of FP16 arithmetic.



GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	87.73
Memory Throughput [%]	87.73
L1/TEX Cache Throughput [%]	87.74
L2 Cache Throughput [%]	3.76
DRAM Throughput [%]	3.20

d) Does this optimization synergize with any other optimizations? How?

I think FP16 synergizes well with streams, tensor cores, cuBLAS, and shared memory tiling. For streams, converting data to FP16 halves the memory transfer size, which enables launching more concurrent streams without saturating memory bandwidth, allowing better overlap of compute and copy operations. For tensor cores, FP16 is the required input type for mixed-precision acceleration; using __half2 unlocks fused multiply-add operations (hfma2) that run on tensor core hardware, significantly speeding up matrix operations. It synergizes with cuBLAS, for cuBLAS provides optimized FP16 GEMM. For shared memory tiling, FP16 halves the data size in shared memory, allowing us to tile larger blocks or fit more tiles per SM, which improves occupancy and reduces memory latency.

e) List your references used while implementing this technique. (you must mention textbook pages at the minimum)

Mixed-Precision Programming with CUDA 8 | NVIDIA Technical Blog

4.2. Half Arithmetic Functions — CUDA Math API Reference Manual 12.9 documentation

4.5. Half Precision Conversion and Data Movement — CUDA Math API Reference Manual 12.9 documentation

4.6. Half2 Arithmetic Functions — CUDA Math API Reference Manual 12.9 documentation

15.2. __half2 — CUDA Math API Reference Manual 12.9 documentation

15.1. __half — CUDA Math API Reference Manual 12.9 documentation